

Project-Based Case Study

Design and Evaluation of the Technology Template “Template-Projekte”

Technical Investigation of a Reusable Foundation for Web, Cloud, and Desktop Projects

Project: Template-Projekte
Author: Marcel Tenhaft
Document version: 9 August 2026

Independently authored technical documentation and evaluation
of the “Template-Projekte” project

Contents

List of Figures	IV
List of Tables	V
List of Abbreviations	VI
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Research Questions	1
1.5 Methodology	2
1.6 Structure of the Case Study	3
2 Requirements for the Technology Template	3
2.1 Target Vision for the Technology Template	3
2.2 Requirements for Reusability and Standardization	4
2.3 Requirements for Web, Cloud, and Desktop Applications	4
2.4 Quality and Maintainability Requirements	4
2.5 Scope	5
3 Architecture of the Technology Template	5
3.1 Overall Architecture	5
3.2 Frontend with Vite and TypeScript	7
3.3 Backend with FastAPI	7
3.4 Desktop Integration with Tauri and Rust	8
3.5 Shared Contracts and Interfaces	8
3.6 Configuration Model	9
3.7 Persistence with PostgreSQL, SQLAlchemy, and Alembic	9
4 Project Profiles and Feature Model	10
4.1 Basic Principle of the Profile Model	10
4.2 Web-only Profile	12
4.3 Web-cloud Profile	12

4.4	Desktop-local Profile	13
4.5	Desktop-cloud Profile	13
4.6	Full-platform Profile	13
4.7	Optional Capabilities	13
4.8	Single-Repository Strategy	14
5	Tooling and Development Workflow	14
5.1	Role of control.py	14
5.2	Project Initialization and Generation	15
5.3	System Check and Installation	16
5.4	Development Mode	17
5.5	Tests and Builds	18
5.6	Release and Packaging Workflow	18
5.7	Developer Onboarding	19
6	Quality Assurance and Verification	20
6.1	Test Strategy	20
6.2	Continuous Integration	20
6.3	Acceptance and Technical Verification	21
6.4	Reproducibility	22
6.5	Security and Configuration Boundaries	22
6.6	Outstanding External Checks	23
7	Evaluation of the Technology Template	23
7.1	Productivity Effects	23
7.2	Strengths	24
7.3	Weaknesses and Limitations	24
7.4	Maintenance Effort	26
7.5	Comparison with Alternative Template Strategies	26
7.6	Overall Evaluation	27
8	Application Scenarios and Transfer to Customer Projects	28
8.1	Suitable Project Types	28
8.2	Unsuitable and Conditionally Suitable Project Types	29
8.3	Analysis of Customer Requirements	30

8.4	Selecting the Project Profile	31
8.5	Generating a Customer Project	31
8.6	Transition to Product Development	32
9	Conclusion	33
9.1	Summary of Results	33
9.2	Answers to the Research Questions	33
9.3	Critical Reflection	34
9.4	Outlook	34
9.5	Conclusion	35
	References	36

List of Figures

1	Methodological steps of the case study	3
2	Overall architecture of the technology template	5
3	Components and responsibility boundaries in the Master Repository	6
4	Profile-dependent runtime architecture	6
5	Provider-neutral deployment architecture	8
6	Configuration precedence and security boundaries	9
7	Deriving a product project from a shared template baseline	10
8	Decision flow for profile and subsequent capability selection	11
9	Structural feature allocation of the five profile presets	12
10	Implemented dependencies in the feature catalog	13
11	Command groups of the central project tooling	15
12	Workflow for profile-driven project generation	16
13	Basic structure of the development workflow	17
14	Cross-platform model for desktop releases	19
15	Evidence chain for technical verification	20
16	Profile verification by evidence state and commit	21
17	Qualitative suitability overview by architectural fit and adaptation effort	29
18	Vertical transfer process from customer requirement to product project	32

List of Tables

1	Key requirements for the technology template	4
2	Requirements matrix for the supported project types	4
3	Declared and resolved technology stack in the examined commit	7
4	Declared baseline features and resulting runtime directories	12
5	Profile-specific technical verification	22
6	Outstanding and external verification items on a4487ac	23
7	Evidence-based comparison of strengths and limitations	25
8	Qualitative comparison of template strategies	27
9	Consolidated evaluation of the technology template	27
10	Mapping of technical project types to the template baseline	29
11	Compact checklist for technology-neutral requirements analysis	30

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AWS	Amazon Web Services
CI	Continuous Integration
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DB	Database
DEB	Debian package format
E2E	End-to-End
GCP	Google Cloud Platform
HPC	High Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PG	PostgreSQL
PID	Process Identifier
RBAC	Role-Based Access Control
SDK	Software Development Kit
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
ZIP	Compressed archive format

1 Introduction

1.1 Background

New software projects repeatedly face the same technical decisions. These concern architecture, technologies, interfaces, configuration, testing, build, and deployment. If these foundations are created anew each time, projects begin from differing technical baselines. At the same time, web, cloud, and desktop applications require both shared structures and platform-specific extensions.

The “Template-Projekte” project consolidates these foundations in a reusable full-stack technology template. It is intended to provide a common basis for web, cloud, and desktop projects (kleiveist, 2026k). Such a basis can consolidate recurring decisions and establish a consistent development model. To achieve this, shared components must be clearly separated from platform-specific extensions.

1.2 Problem Statement

Repeatedly initializing similar projects consumes effort outside product development. Without a shared baseline, structure, tools, configuration, and build processes differ; separate starting points increase maintenance effort and may diverge technically.

The template must therefore combine standardization with flexibility: a common basis should enable reuse without burdening projects with unnecessary components. This case study examines how project profiles and optional extensions address this trade-off and where adaptation is still required (kleiveist, 2026f, 2026g).

1.3 Objectives

The case study examines the architecture, shared and profile-specific components, and the development and quality-assurance workflows of the “Template-Projekte” project (kleiveist, 2026f, 2026l).

Based on existing project artifacts and evidence, it assesses reusability, standardization, productivity potential, and suitability for different project types. From this assessment, it derives criteria for use and selection, technical limitations, and external prerequisites (kleiveist, 2026j). The study does not investigate universal suitability for every class of software or a product-specific domain architecture.

1.4 Research Questions

Six questions structure the investigation and assessment:

1. Which functional and technical, quality, and operational requirements does the technology template address?
2. How do the architecture and profile model support the provision of different variants for web, cloud, and desktop projects?

3. To what extent do the shared technical core and the single-repository strategy promote reuse and standardization across projects?
4. What potential does the template offer for reducing recurring effort in project initialization, development workflows, quality assurance, and deployment?
5. For which project types and customer requirements does the template provide a suitable technical starting point?
6. Which technical limitations, maintenance efforts, risks, and external dependencies must be considered when using it?

Their answers are based on requirements, architecture, and the available verification evidence.

1.5 Methodology

The investigation follows a technical case-study approach (Runeson & Höst, 2009). An inventory records the structure, components, and boundaries of responsibility within the master repository. The subsequent architecture analysis examines the frontend, backend, desktop integration, interfaces, persistence, and deployment. It distinguishes substantiated properties from assumptions and unresolved verification points (kleiveist, 2026f).

The profile analysis examines the shared core, variants, and optional capabilities. In addition, the Python tooling is examined with respect to generation, system checks, installation, development, testing, builds, and release (kleiveist, 2026g, 2026l). Internal project descriptions of the architecture, profiles, and tools serve as primary sources for the documented state.

Existing tests, build results, and documented acceptance results are mapped to the requirements and architecture decisions. External or product-specific checks remain separate. This evidence underpins the assessment and the transfer of findings to potential customer projects (kleiveist, 2026j). Figure 1 shows the sequence of the investigation. Assessment and transfer therefore take place only after the inventory, analysis, and verification; unresolved verification points are not treated as confirmed properties.

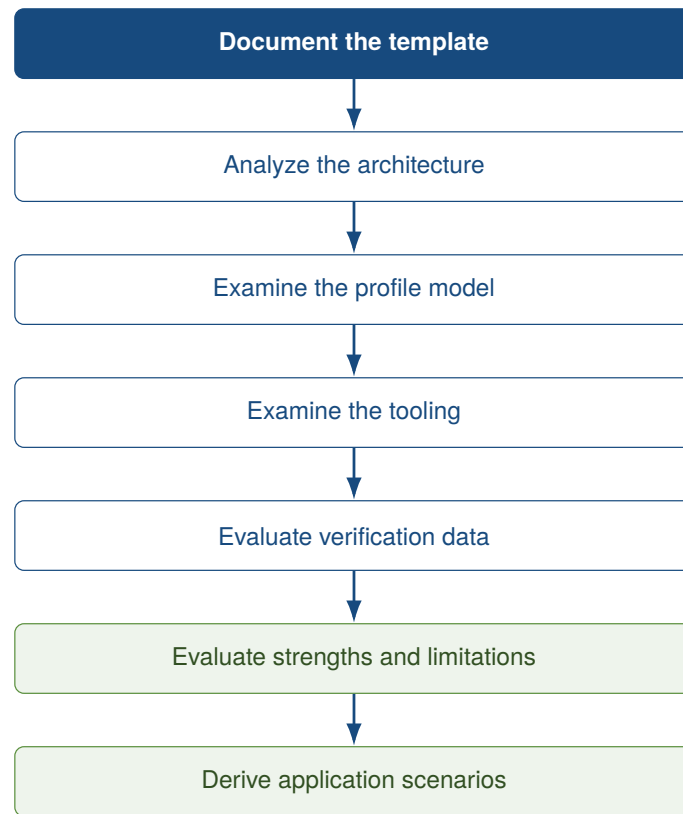


Figure 1: Methodological steps of the case study

Source: Author's own illustration.

1.6 Structure of the Case Study

The requirements are followed by the architecture, project profiles, and tooling. The case study then addresses quality assurance and verification, assesses the observed state, and transfers the findings to customer projects. The conclusion answers the research questions and identifies limitations and prospects for further development.

2 Requirements for the Technology Template

2.1 Target Vision for the Technology Template

The template is intended to provide a reusable basis for web, cloud, and desktop projects. To this end, it combines planned commonality with controlled variability (Clements & Northrop, 2001; Krueger, 1992). A shared core is intended to centralize structure, workflows, tooling, documentation, configuration, tests, and interfaces.

Profiles select suitable combinations from this core; optional capabilities such as PostgreSQL extend compatible backend profiles (kleiveist, 2026g). The frontend, backend, desktop layer, persistence, tooling, and deployment remain separate areas of responsibility. At this stage, these statements describe requirements, not verification results.

2.2 Requirements for Reusability and Standardization

The baseline is intended to define shared structures and public workflows while allowing controlled profile variants. Centralized maintenance prevents copies from diverging. Table 1 summarizes the verifiable requirements.

Table 1: Key requirements for the technology template

ID	Requirement	Operationalized objective	Basis for subsequent verification
A-01	Baseline	It must be possible to generate the basic structure.	Project generation
A-02	Structure	Responsibilities are assigned to consistent locations.	Repository structure
A-03	Profiles	Only designated components are activated.	Profile model
A-04	Workflows	Checks, initialization, installation, startup, testing, and building share common entry points.	Tooling
A-05	Maintainability	Shared components are maintained centrally.	Repository strategy
A-06	Traceability	Changes and configuration are versioned and documented.	Version state, documentation
A-07	Extensibility	Optional capabilities have documented dependencies.	Capability model

Source: Author's own compilation.

The stated verification basis is subsequently used to assess these target criteria.

2.3 Requirements for Web, Cloud, and Desktop Applications

All variants are intended to share the frontend foundation, structure, configuration logic, tests, and documentation. Browser projects do not require a desktop layer and need a backend only when necessary. Cloud variants require an API, deployment boundaries, and health and readiness checks. Desktop variants use the same frontend in a native runtime; combined variants add a backend and, where applicable, browser access.

Table 2: Requirements matrix for the supported project types

Requirement	Web	Cloud	Local desktop	Desktop + Cloud
Frontend	required	required	required	required
Backend / API	optional	required	not inherently required	required
Desktop runtime	no	no	required	required
Deployment boundary	optional	required	no	required
Persistence	optional	optional	product-specific	optional

Source: Author's own compilation based on kleiveist (2026g).

PostgreSQL is only an optional capability for backend-enabled profiles, not a separate platform profile (kleiveist, 2026g). Public client configuration must be separated from server-side values, including secrets; no particular cloud platform is assumed.

2.4 Quality and Maintainability Requirements

The requirements are guided by verifiable characteristics of ISO/IEC 25010 (ISO/IEC, 2023). Components and profiles should have clear responsibilities, be testable in isolation, and remain maintainable

through documented decisions. Extensions and technology updates should not require a copy of the entire baseline, although they still entail risks.

Identical inputs and configurations should produce traceable generation and build results. This strengthens the verifiable relationship between the source-code state and the artifact (Lamb & Zacchiroli, 2022). It should be possible to generate target artifacts in a controlled manner, and configuration priorities should be comprehensible; server-side secrets remain separate from client configuration.

2.5 Scope

The template is not a universal software platform. Its target scope encompasses recurring web, back-end, and desktop projects, including full-stack business applications.

Business rules, domain and customer data models, and product-specific authentication, role, and authorization concepts are not part of the baseline. No cloud provider is prescribed (kleiveist, 2026f).

Highly platform-specific native applications, games, and highly specialized real-time systems fall outside the immediate target scope.

3 Architecture of the Technology Template

3.1 Overall Architecture

The baseline consists of a master repository with a shared core and declarative variants. Profiles select reusable features; optional capabilities extend only compatible profiles. This controlled variability avoids separate copies of the template (Clements & Northrop, 2001; kleiveist, 2026g).

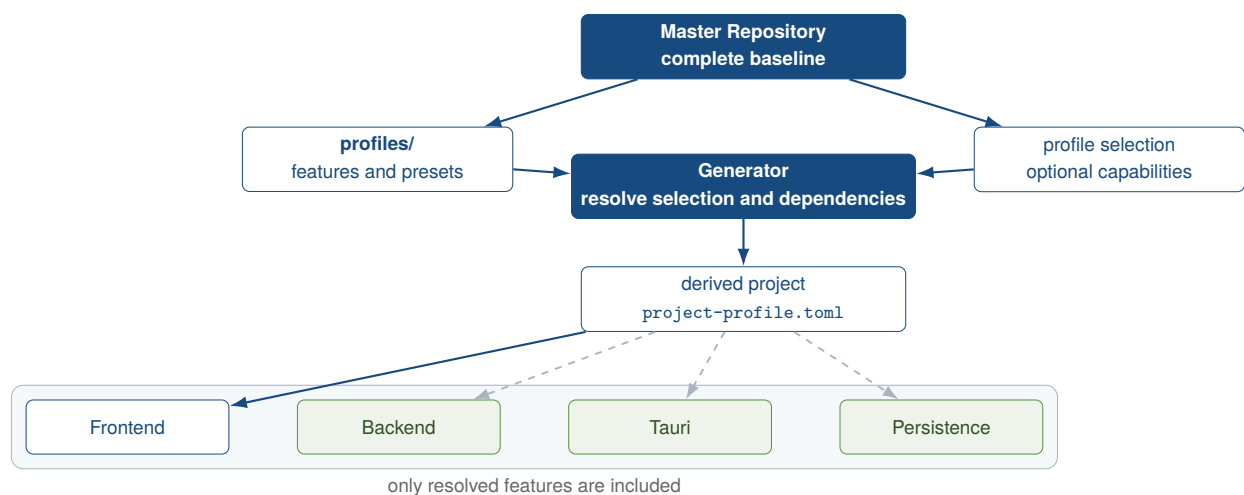


Figure 2: Overall architecture of the technology template

Source: Author's own illustration based on kleiveist (2026f) and kleiveist (2026g).

Figure 2 distinguishes the baseline from the derived project. The generator resolves the profile and capabilities, copies the associated paths, and records the result in `project-profile.toml`. The frontend, backend, Tauri, and persistence remain independent components, some of which are optional

(kleiveist, 2026f).

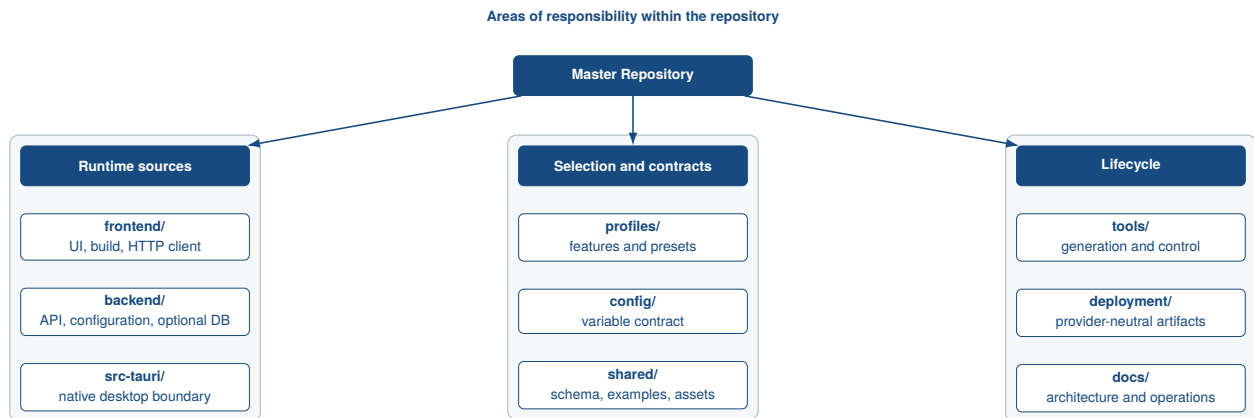


Figure 3: Components and responsibility boundaries in the Master Repository

Source: Author's own illustration based on kleiveist (2026f).

The separation of components in Figure 3 keeps the client, server, native integration, and shared artifacts distinct. Profiles and configuration describe the structure and runtime values; tooling and deployment control the components outside the product runtime.

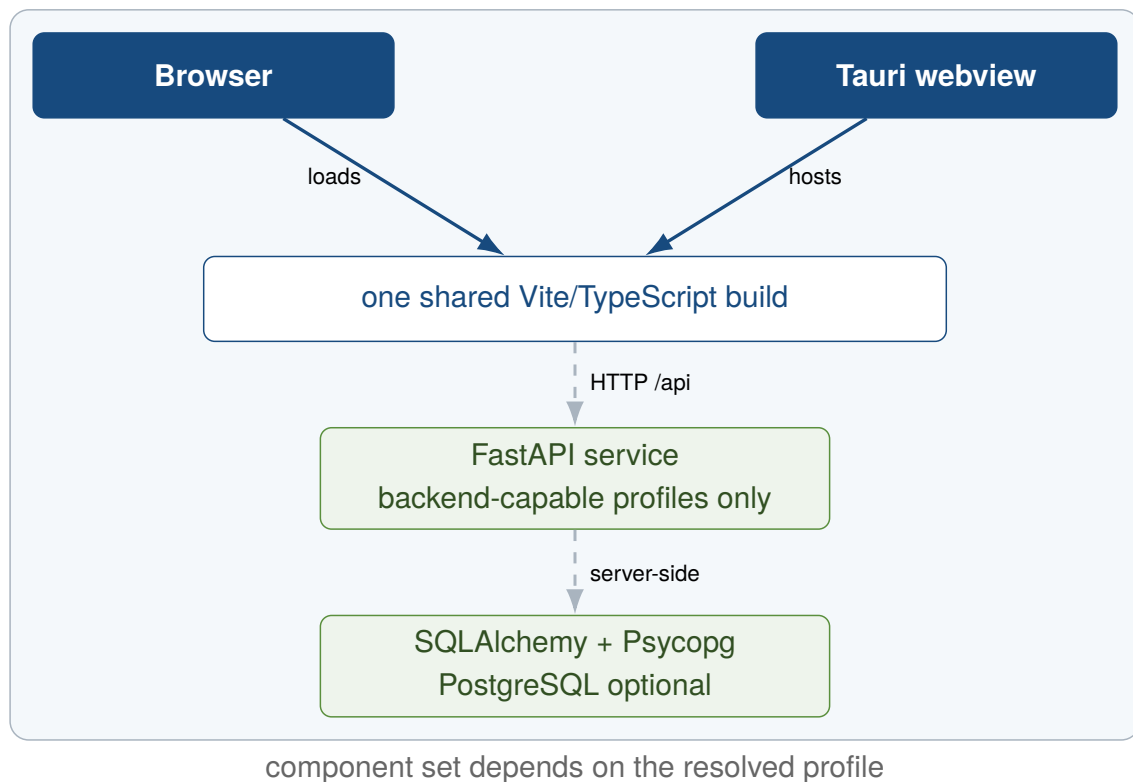


Figure 4: Profile-dependent runtime architecture

Source: Author's own illustration based on kleiveist (2026f) and kleiveist (2026g).

At runtime, either the browser or a Tauri webview loads the same Vite build (Figure 4). Backend-enabled profiles communicate with FastAPI over HTTP; only the backend accesses persistence when it is enabled. This keeps the client, server, and data storage separately testable and extensible.

3.2 Frontend with Vite and TypeScript

The frontend is the shared presentation layer for all five profiles. Vite is used for development and building; TypeScript checks the strictly configured source code, while Vitest tests the HTTP client (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

The generated profile module enables backend access only in appropriate variants. The browser and Tauri use the same source files and a public `VITE_API_BASE_URL`. Server logic, secrets, and direct database access remain outside the client; the web-only and desktop-local profiles require no backend (kleiveist, 2026f, 2026g).

Table 3 documents the areas and versions in the commit under examination.

Table 3: Declared and resolved technology stack in the examined commit

Technology	Role in the template	Version / source	Project evidence
Vite	development server and frontend build	range from 6.0.7; lock: 6.4.2	frontend/package.json, frontend/package-lock.json
TypeScript	static checking of the frontend	range from 5.7.3; lock: 5.9.3	frontend/package.json, frontend/tsconfig.json
FastAPI	HTTP API and validation	≥ 0.111 , < 1 ; production: 0.111.0	backend/requirements.txt, backend/requirements-production.txt
Tauri	desktop webview and packaging boundary	major version 2; lock: 2.11.5	src-tauri/Cargo.toml, src-tauri/Cargo.lock
Rust	native composition layer	at least 1.77; edition 2021	src-tauri/Cargo.toml
PostgreSQL	optional relational runtime unit	container: 16.4-alpine3.20	deployment/compose.yaml
SQLAlchemy	engine, session, and model foundation	≥ 2 , < 3 ; production: 2.0.36	backend/requirements-database*.txt
Alembic	explicit schema migrations	≥ 1.13 , < 2 ; production: 1.14.0	backend/requirements-database*.txt
Psycopg	PostgreSQL driver	≥ 3.2 , < 4 ; production: 3.2.3	backend/requirements-postgres*.txt

Source: Author's own compilation based on the examined project state (kleiveist, 2026k).

3.3 Backend with FastAPI

The backend forms a separate HTTP layer for the web-cloud, desktop-cloud, and full-platform profiles. Under `/api`, FastAPI currently provides only `/health` and `/ready`. Product-specific services, domain models, and authentication are not prescribed (kleiveist, 2026f; Ramírez, 2026).

Liveness reports the process state. When the database capability is active, readiness additionally checks the connection and responds with HTTP 503 on failure without disclosing internal details. The router, settings, and dependency are separately testable; service and domain modules are added only by the product.

The units remain separate in deployment (Figure 5): the Vite build can be deployed as static content, while FastAPI runs in its own unprivileged container. Compose represents this provider-neutral boundary locally; PostgreSQL is added only when explicitly selected (kleiveist, 2026e).

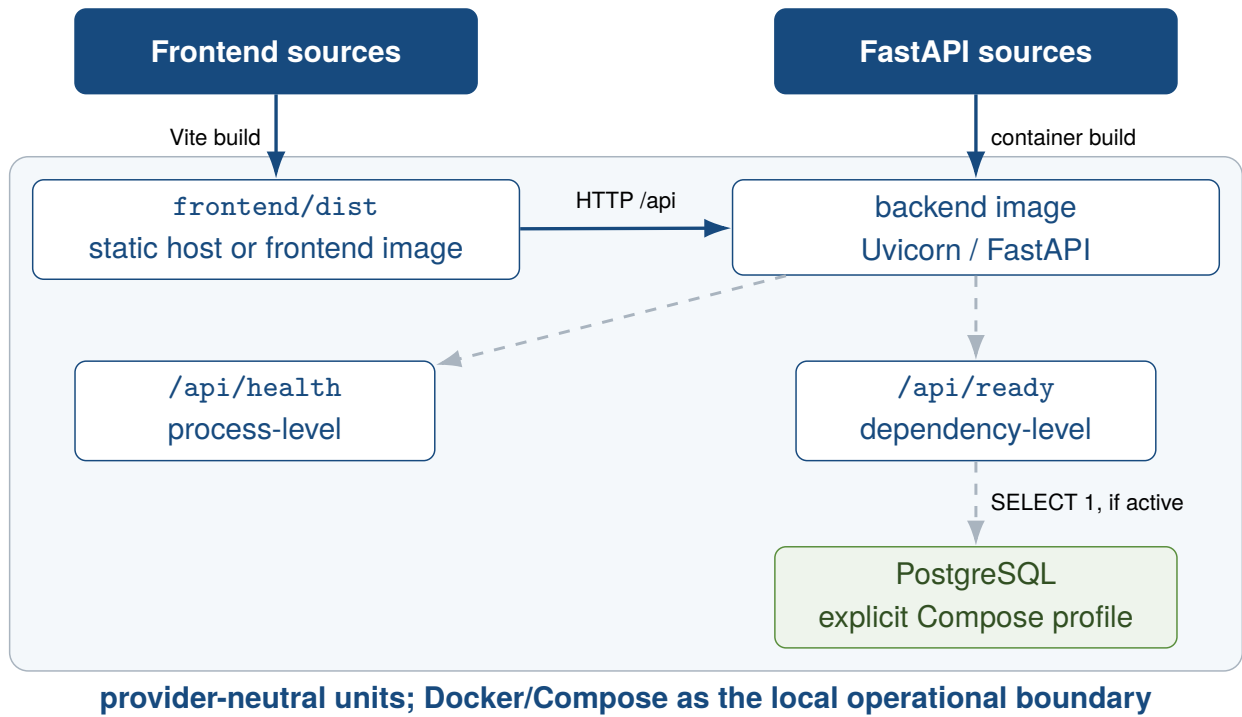


Figure 5: Provider-neutral deployment architecture
Source: Author's own illustration based on kleiveist (2026e).

3.4 Desktop Integration with Tauri and Rust

Tauri provides a native boundary without duplicating the frontend. During development, it uses the Vite server; in the build, it uses `frontend/dist`. The Rust entry point only creates the Tauri application; native domain commands and a FastAPI sidecar are absent from the baseline (Klabnik et al., 2026; kleiveist, 2026f; Tauri Contributors, 2026b).

The default capability grants only `core:default`; a Content Security Policy restricts resources and development endpoints. Additional native permissions must be added on a product-specific basis (Tauri Contributors, 2026a). In the desktop-local profile, local functionality resides in the frontend or in future Tauri commands; desktop-cloud and full-platform use the public FastAPI URL. Packaging, signing, and the choice between a remote API, sidecar, or local commands remain product decisions.

3.5 Shared Contracts and Interfaces

`shared/` is included in every generated project. It contains static assets, a JSON Schema conforming to Draft 2020-12, and valid and invalid test examples. The contents are serializable and framework-neutral, but define neither a product API nor executable framework code (kleiveist, 2026f, 2026k).

However, the runtime does not use the schema automatically: neither the frontend nor the backend imports it. The health contract is likewise mirrored manually as a TypeScript interface and a FastAPI response. Without code generation or synchronization, contract changes must be propagated on both sides and in consumer tests. The documented target role of shared contracts is therefore not yet fully implemented.

3.6 Configuration Model

Project structure and runtime configuration are separate. The project manifest determines the features and is stored in `project-profile.toml`. The variable contract is defined in `config/environment.toml`. For active features, the priority order is CLI override, process environment, local `.env`, and contract default. Derived values also remain traceable (kleiveist, 2026i).

Figure 6 shows the security boundaries. Vite receives only the approved `VITE_API_BASE_URL`; FastAPI processes server values with explicit types and masks `DATABASE_URL`. The tooling forwards only relevant values and removes known secret names from the Tauri environment. Invalid or missing values result in controlled termination without exposing secrets.

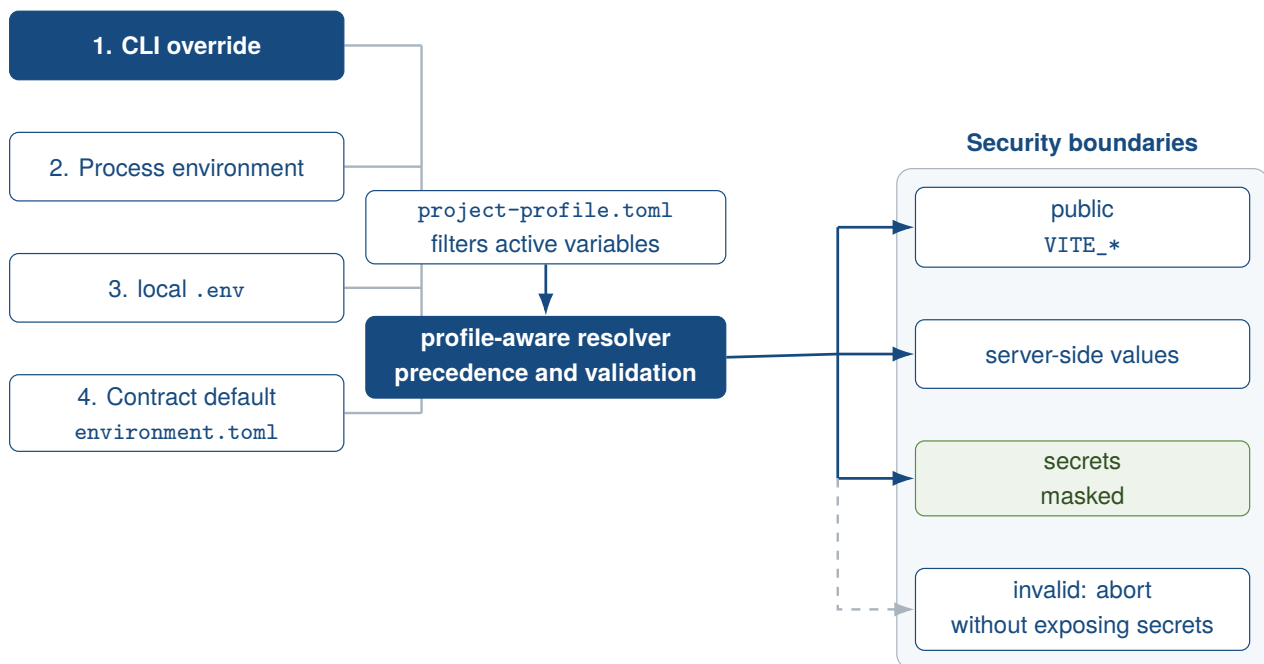


Figure 6: Configuration precedence and security boundaries

Source: Author's own illustration based on kleiveist (2026i).

3.7 Persistence with PostgreSQL, SQLAlchemy, and Alembic

PostgreSQL is optional and is not a separate profile. `postgres` requires database and thus a backend; the web-only and desktop-local profiles are excluded. The database capability comprises the SQLAlchemy engine, sessions, and Alembic. PostgreSQL adds Psycopg 3, URL requirements, and integration tests and remains a separate runtime unit (kleiveist, 2026d; PostgreSQL Global Development Group, 2026).

The engine is created only upon access and validates connections before use; sessions are closed in a controlled manner. The empty declarative `Base` is an extension point for domain models (SQLAlchemy Authors and Contributors, 2026).

Alembic uses the same metadata basis for online and offline migrations. Without domain models, there is no initial revision. Neither `create_all()` nor automatic migrations at FastAPI startup are implemented; changes require explicit Alembic commands (Bayer, 2026; kleiveist, 2026d). `/api/health` remains independent of the database, while `/api/ready` checks reachability when persistence is ac-

tive.

4 Project Profiles and Feature Model

4.1 Basic Principle of the Profile Model

The master repository manages variants through features, thereby supporting systematic reuse without constituting a complete software product line (Clements & Northrop, 2001; Krueger, 1992). Here, *Core* denotes the paths included in every product project. A *Feature* groups the paths and dependencies of a technical capability, while a *profile* represents a permissible combination of features. A *capability* optionally extends a profile and is not itself a platform profile. The *manifest* records the selection in the generated project (kleiveist, 2026g, 2026k). The Core includes version information, configuration, documentation, profile descriptions, shared artifacts, and tooling, among other elements. Technical runtime components, by contrast, reside in the selected feature paths.

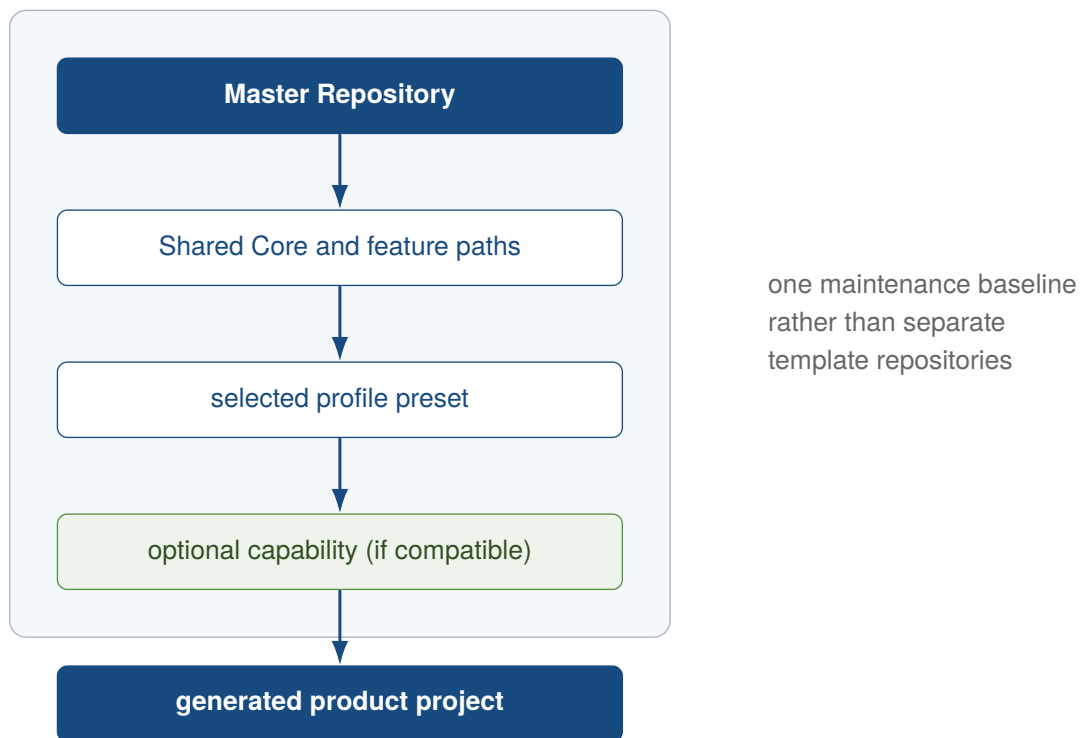


Figure 7: Deriving a product project from a shared template baseline

The generator resolves the profile and compatible capabilities and copies the Core and active feature paths into a new product project (Figure 7). Inactive directories are not selected; web profiles additionally omit the Tauri CLI. The name, slug, and, for Tauri, the application ID can be customized (kleiveist, 2026g, 2026l). Resolution produces an ordered scaffold plan. As a result, the target project does not merely contain disabled components; it predominantly contains only the directories intended for its variant.

Backend and cloud occur together in the current presets. For desktop projects, an additional browser channel distinguishes `full-platform` from `desktop-cloud`. Capabilities are checked only after the profile has been selected; the resolver does not silently add missing platform features (Figure 8).

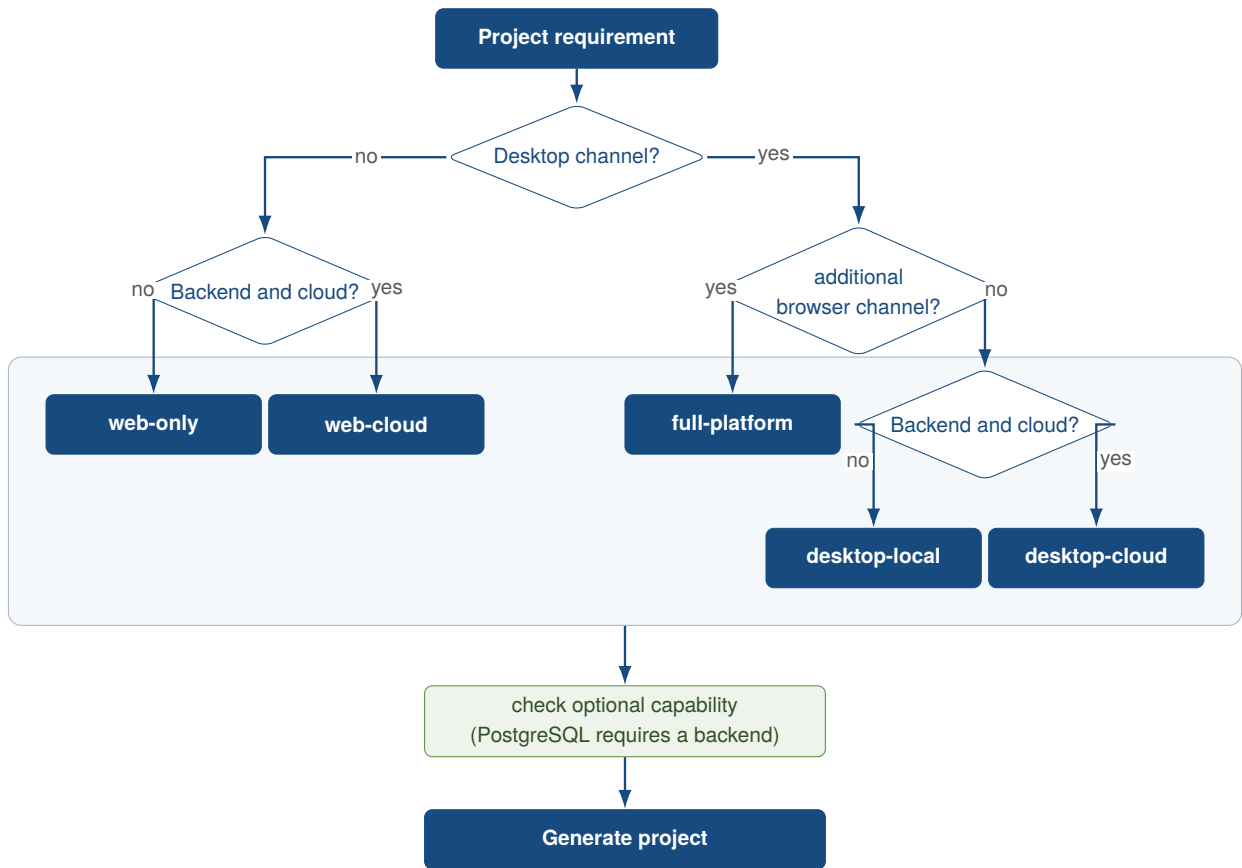


Figure 8: Decision flow for profile and subsequent capability selection

The generated `project-profile.toml` records the schema version, profile identity, requested extensions, and fully resolved features. The tooling validates it and handles only active services and workflows. The master manifest uses `full-platform + postgres` without modifying the preset itself. Runtime values, ports, URLs, and secrets instead belong to the separate configuration model (kleiveist, 2026i, 2026k). Requested optional features and the resolved result remain separately visible in the manifest. This distinction makes transitive dependencies traceable.

Figure 9 and Table 4 compare the five presets. None enables PostgreSQL by default.

Profile	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	included	–	–	–	incompatible
web-cloud	included	included	–	included	optional
desktop-local	included	–	included	–	incompatible
desktop-cloud	included	included	included	included	optional
full-platform	included	included	included	included	optional

Figure 9: Structural feature allocation of the five profile presets

Source: Author's own illustration based on the feature catalog and the profile definitions in the examined project state (kleiveist, 2026g, 2026k).

Table 4: Declared baseline features and resulting runtime directories

Profile	Baseline features	Runtime directories	PostgreSQL
web-only	frontend	frontend/	incompatible
web-cloud	frontend, backend, cloud	frontend/, backend/, deployment/	optional
desktop-local	frontend, tauri	frontend/, src-tauri/	incompatible
desktop-cloud	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional
full-platform	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional

4.2 Web-only Profile

`web-only` adds only the Vite frontend to the Core. The browser loads it without a FastAPI, desktop, deployment, or database layer; the Tauri script and CLI are removed. The profile is therefore suitable for pure client applications without backend access (kleiveist, 2026g). Compared with `web-cloud`, it consequently lacks the API and cloud feature; the generated frontend also does not enable a backend status request.

4.3 Web-cloud Profile

`web-cloud` combines a Vite frontend, a separate FastAPI backend, and provider-agnostic deployment files. It has no native desktop layer (kleiveist, 2026e, 2026g). The database module, Alembic, and Psycopg are added only with the compatible PostgreSQL capability, which must be selected separately. The frontend accesses the separate backend through a public URL. Without the capability, the profile remains database-free despite its cloud designation.

4.4 Desktop-local Profile

`desktop-local` runs the shared frontend in a Tauri webview. Backend, deployment, and database paths are absent (kleiveist, 2026g). Thus, *local* denotes the absence of a cloud API, not the presence of an embedded SQL database. Because database requires a backend, PostgreSQL is incompatible. Local domain functionality must therefore reside in the frontend or in Tauri commands added for the specific product.

4.5 Desktop-cloud Profile

`desktop-cloud` connects the Tauri frontend to the separate FastAPI backend through a public URL and adds deployment files. PostgreSQL is compatible but remains optional (kleiveist, 2026e, 2026g). The technical base features correspond to `full-platform`; the profile ID, however, identifies the desktop client as the intended product channel. The distinction is semantic: it does not introduce an additional technical component.

4.6 Full-platform Profile

`full-platform` provides the browser and Tauri as two product channels for the same frontend base; both access the same FastAPI backend. Its scaffold differs from `desktop-cloud` only in profile identity and description, not in its base features (kleiveist, 2026g, 2026k). The browser build and desktop application thus use the same frontend and backend base.

Full does not include persistence. The database capability is added only by `postgres`.

4.7 Optional Capabilities

The catalog contains six features. `tauri` requires `frontend`; `cloud` and `database` require `backend`; and `postgres` requires `database`. The directed rules in Figure 10 do not describe runtime communication (kleiveist, 2026d, 2026g).

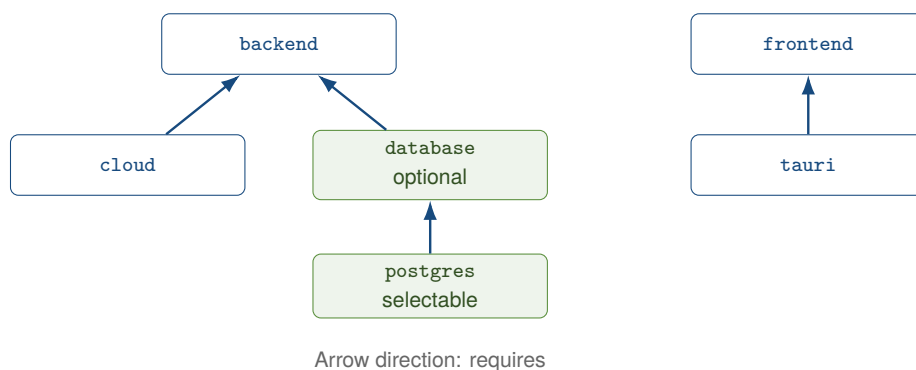


Figure 10: Implemented dependencies in the feature catalog

Only `postgres` is currently marked as a selectable capability. For backend profiles, it first resolves the optional, internal-only database feature and adds SQLAlchemy, Alembic, Psycopg, and tests.

The PostgreSQL service is already declared as an inactive profile in the Compose document; the capability adds backend infrastructure and dependencies (kleiveist, 2026e, 2026k). Thus, `web-cloud + postgres` consists of the frontend, backend, cloud, database base, and PostgreSQL extension. The selection therefore does not merely activate the Compose entry; it adds the associated backend paths.

The resolver adds only optional prerequisites. It therefore rejects `web-only + postgres` and `desktop-local + postgres` because both lack a backend. Declaring new catalog entries alone does not make them implemented. They also require their own paths, dependencies, and tests.

Two implementation discrepancies remain: the dialog offers only `selectable` PostgreSQL, but a direct `-with database` argument accepts the database capability that is optional for backend profiles. In addition, contrary to the general profile documentation, entries for `.env.example` originate from `config/environment.toml`, not from `features.toml`. The presets remain unchanged, but the distinction between internal and selectable capabilities is not fully enforced.

4.8 Single-Repository Strategy

The five profiles are not template forks. The Core, feature implementations, profiles, and generator reside in a single maintenance base from which independent product projects are created (kleiveist, 2026g, 2026k).

The master also contains optional implementations and itself enables PostgreSQL. A generated project, by contrast, receives only the Core, resolved feature paths, manifest, frontend profile, and an appropriate `.env.example`. It then remains separate from the unchanged master. Centralized maintenance reduces parallel baselines; generated projects are not automatically synchronized back to the master. This separation protects the maintenance base from product changes. Conversely, existing generated products do not benefit from subsequent fixes to the master unless those fixes are incorporated deliberately.

5 Tooling and Development Workflow

5.1 Role of `control.py`

`tools/control.py` consolidates the public workflows. Its commands cover initialization, diagnostics, installation, development, configuration, database operations, tests, builds, containers, Tauri, versioning, releases, and documentation. Help and group views structure this interface (kleiveist, 2026a, 2026l). The principal entry points include `init`, `doctor`, `install`, `run`, `test`, and `build`. The `db`, `tauri`, `container`, and `release` groups add only the specialized workflows required in each case.

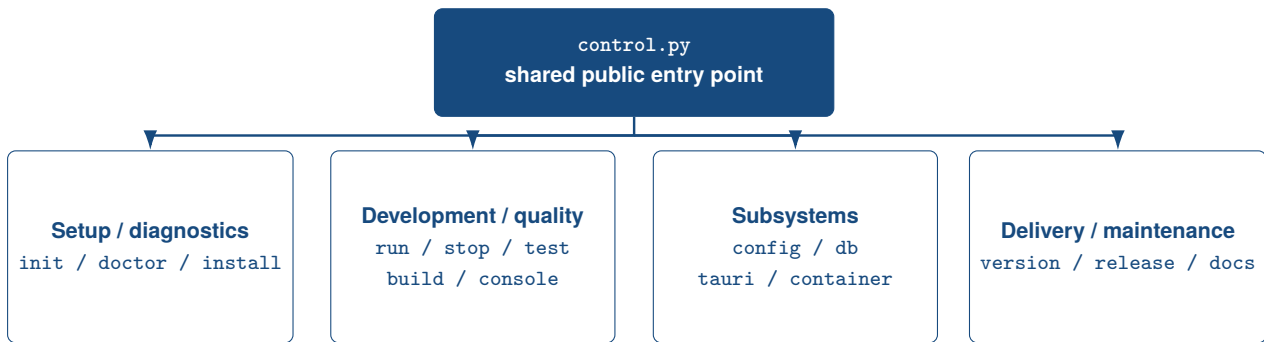


Figure 11: Command groups of the central project tooling

Source: Author's own illustration based on kleiveist (2026a) and kleiveist (2026l).

The handlers load the manifest and, when required, the runtime configuration, then delegate to the respective specialized tools. These tools remain independent. The guided `console` invokes the same commands and does not form a second system.

Different exit codes distinguish help and success, domain errors, usage errors, and cancellation. Expected profile and generation errors are shown without a traceback; unexpected exceptions are logged. This convention supports interactive diagnostics and CI evaluation (kleiveist, 2026a, 2026c). In particular, successful execution, domain failure, and incorrect usage can be distinguished programmatically. Warnings do not automatically block the workflow.

5.2 Project Initialization and Generation

`init` creates an independent product project from a profile and compatible capabilities. Selection can be guided or non-interactive; project identity, target, and dry run are configurable. A modified Tauri identity requires a valid reverse-domain identifier (kleiveist, 2026g, 2026l).

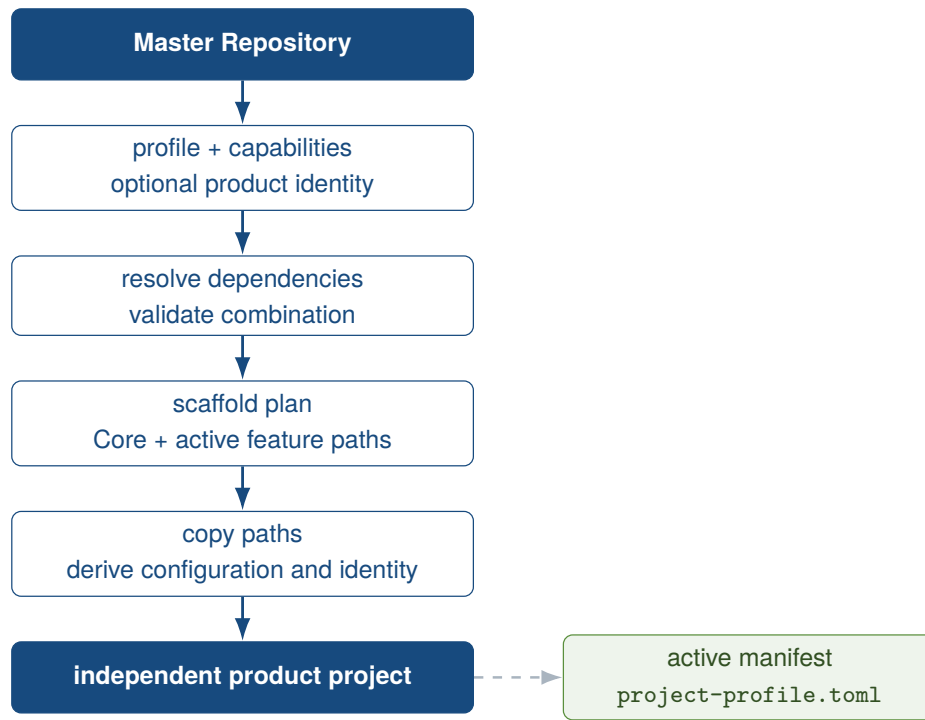


Figure 12: Workflow for profile-driven project generation

Source: Author's own illustration based on kleiveist (2026g) and kleiveist (2026l).

The generator resolves dependencies, creates the scaffold plan, and validates the sources and target before writing. It copies the Core and active features, generates the manifest, frontend profile, and `.env.example`, and adjusts existing product metadata. Transient directories are omitted; web profiles omit the Tauri script and CLI. When a product identity is specified, only frontend, backend, Compose, Tauri, and Cargo metadata that are actually present are adjusted. The scope thus remains dependent on the selected profile.

Within the master, targets are permitted only under `.generated/`; external targets are also possible. The repository itself and nonempty targets are rejected, and `-dry-run` writes nothing. One limitation remains: only the dialog filters by `selectable`; a `direct -with database` argument is accepted for backend profiles (kleiveist, 2026k). These preliminary checks keep the master and target separate and prevent partially generated projects in known conflict cases. The derivation is designed to be deterministic; nevertheless, the empirical evidence of reproducibility remains limited to the case examined.

5.3 System Check and Installation

The read-only `doctor` checks runtimes, configuration, paths, dependencies, and ports according to the profile. Disabled layers are not an error. In general, missing Docker produces only a warning; container, database, and Tauri prerequisites have their own stricter diagnostic paths (kleiveist, 2026i, 2026l).

`install` sets up only active components: frontend dependencies are preferentially installed from the lockfile, and backend dependencies are installed in a virtual environment. Database packages, the tooling environment, and Chromium are added only when required; for Python, a `pip/venv` fallback is

available alongside `uv`. A local `.env` remains unchanged. `tauri install` handles native operating-system and Rust prerequisites separately. Specifically, the frontend uses `npm ci` when a lockfile is present. An active backend receives its own environment and only the profile-dependent requirement sets. Chromium is installed only when Playwright is configured; disabled layers are skipped.

The README requires Git, Python 3.11 or later, Node.js 20 or later, and Rust Stable for desktop profiles. The general Doctor does not check Git or these version ranges; the Tauri Doctor accepts any active Rust toolchain. CI, by contrast, explicitly sets up the specified major versions of Python and Node. Local enforcement therefore differs from the documentation (kleiveist, 2026c, 2026k). This limitation concerns how the prerequisite is checked, not whether the documented prerequisite applies.

5.4 Development Mode

Figure 13 shows the profile-dependent path from diagnostics and installation to quality assurance and delivery.

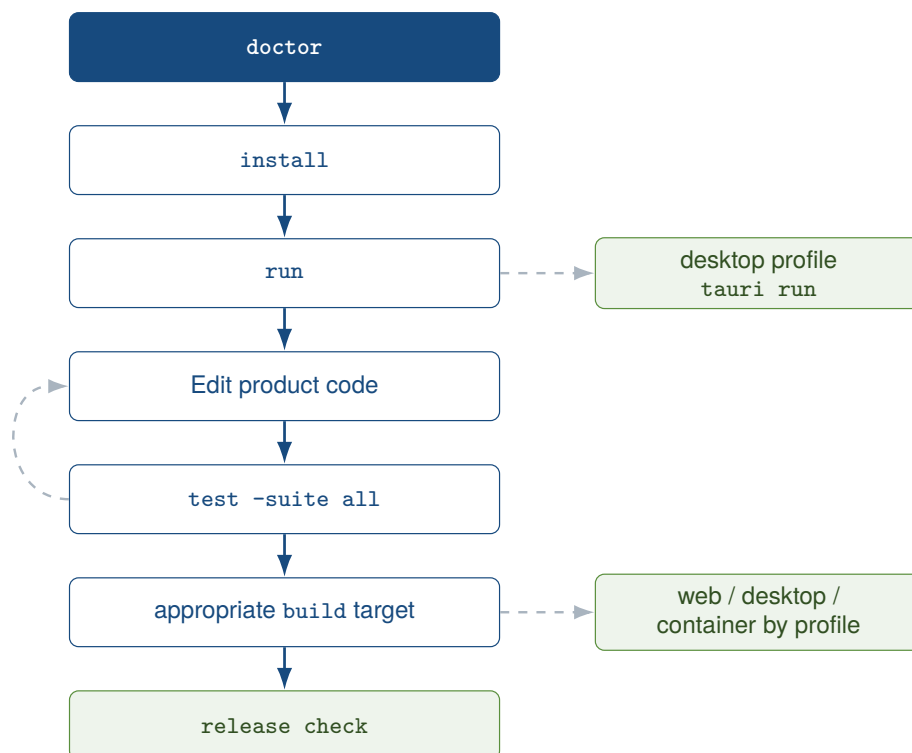


Figure 13: Basic structure of the development workflow

Source: Author's own illustration based on kleiveist (2026l) and kleiveist (2026h).

`run` starts Vite and, only when the backend is active, Uvicorn as well. In the foreground, cancellation terminates the tracked process groups; in detached mode, PID and log data are stored. `stop` terminates tracked processes and, by default, listeners on the configured ports as well; `-tracked-only` restricts this behavior to the tool's own tracked state (kleiveist, 2026a, 2026l). In the foreground, output from active services is combined. Detached mode, by contrast, retains process and log data under `tools/.runtime`; this state store provides the basis for `stop`.

The separate desktop path delegates to `tauri dev` and removes known server-side secrets from the child-process environment. It supports foreground and background operation. The console merely

guides users through the same CLI workflows. The native development path thus remains separate from general Vite/Uvicorn operation without duplicating the command logic.

5.5 Tests and Builds

The eight test suites cover tooling, schema, API, database, PostgreSQL, frontend, E2E, and Tauri/Rust. `test -suite all` provides the complete project-wide entry point; reports are optional (kleiveist, 2026a, 2026l). Among other elements, the tooling suite tests the generator, profiles, and configuration; the component suites delegate to Pytest, Vitest, Playwright, or Cargo. The test areas therefore remain independently executable.

Selection remains profile-dependent: if a feature is absent, the suite reports `SKIP`; if sources or pre-requisites are missing for an active feature, it reports `FAIL`. PostgreSQL without a test URL and E2E without Playwright configuration are skipped. The runner can repair missing backend environments and manage the development services for E2E. This behavior deliberately distinguishes absent features from defective active components. A skipped test is therefore not evidence of success.

The build paths remain separate: web produces Vite output and a ZIP; desktop delegates the target and bundle to Tauri; and container builds frontend, backend, or both images for cloud profiles. `container validate` checks the Compose model without starting it (kleiveist, 2026b, 2026h). The web path additionally creates a ZIP package. Desktop builds remain tied to an active Tauri feature and the respective target platform.

GitHub Actions uses the same public entry points for the Core, profile matrices, PostgreSQL, and desktop targets. CI supplements these with operating systems, runtimes, and a temporary PostgreSQL service (kleiveist, 2026c). Local and external workflows therefore use comparable entry points, but do not run under identical environmental conditions.

5.6 Release and Packaging Workflow

`VERSION` is the central version source; validation and write-based synchronization are separate. `release check` does not produce artifacts. The command checks the version, product identity, Tauri rules, tag, and Git working tree. Inconsistencies, placeholders from the template, and a mismatched or modified state are errors. Missing signing configuration, by contrast, produces only a warning. Only the master repository accepts its canonical template identity (kleiveist, 2026a, 2026h).

Web delivers a Vite build and ZIP. On the respective host, Tauri produces native Windows, macOS, or Linux bundles; additional paths are available for a portable Windows ZIP and a Linux cross-build. CI produces only ephemeral, unsigned validation artifacts. These packages validate the technical packaging path. Without a signature and product-specific approval, they are not publishable releases.

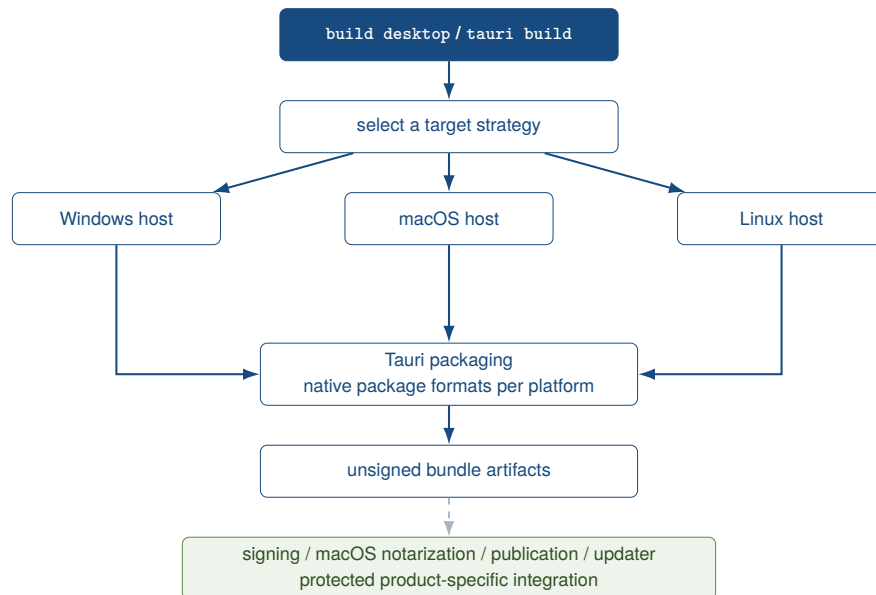


Figure 14: Cross-platform model for desktop releases

Source: Author's own illustration based on kleiveist (2026h).

Signing and notarization are product-specific. The same applies to publication, app stores, and updaters. The release workflow combines tests with web and container builds. These are followed by the gate and the unsigned desktop matrix. The workflow neither publishes nor deploys (kleiveist, 2026c, 2026h).

For cloud profiles, the container commands check Docker, Compose, and deployment files and build versioned images. Compose simulates production locally and optionally runs PostgreSQL. The foundation does not prescribe a provider and replaces neither secrets management nor a controlled migration job (kleiveist, 2026b, 2026e). A Docker/Compose foundation is therefore not equivalent to a complete architecture for AWS, Azure, or GCP.

5.7 Developer Onboarding

Onboarding follows the sequence of cloning the master, running `doctor`, running `init`, moving to the target project, running `doctor` again, and then running `install` and `run`. Product code, configuration, tests, and commits then belong in the derived repository. Template maintenance, by contrast, takes place in the master and validates the entire baseline there (kleiveist, 2026g, 2026k). This separation prevents product development from inadvertently modifying the shared template.

The basic prerequisites are Git, Python 3.11 or later, and Node.js 20 or later. Desktop profiles additionally require Rust Stable and platform-specific Tauri packages; Docker, PostgreSQL, and PyGitIndex are required only for the respective optional tasks. Optional tools therefore do not block every profile.

The README and specialized documents organize getting started, tooling, profiles, configuration, CI, containers, and releases into separate areas. This standardizes the public setup paths, but does not yet demonstrate a measurable reduction in onboarding time (kleiveist, 2026c, 2026h, 2026i, 2026l). The documentation structure thus supplements technical onboarding, but does not replace an empirical measurement of its duration.

6 Quality Assurance and Verification

6.1 Test Strategy

Verification associates every observation with an artifact, an executed check, and a commit-bound result. Test code alone does not demonstrate execution, just as a workflow file does not demonstrate a successful CI run. Execution results and the acceptance report therefore carry more weight than code, configuration, and documentation (ISO/IEC, 2023; Runeson & Höst, 2009).

Four levels cover the baseline: tooling tests check the CLI, profiles, generator, configuration, release, and repository; component tests cover the schema, backend, frontend, and Tauri/Rust. Integration tests exercise PostgreSQL 16 together with the configuration and Alembic. Build checks produce web, container, or native artifacts (kleiveist, 2026c).

The scope follows the active profile. *PASS* denotes a successful check, and *FAIL* an executed check that failed. *Not verified* means that evidence is absent, while *out of scope* denotes a deliberate boundary; disabled items are not applicable. Generation and structure checks address A-01 and A-03, the public tooling interface addresses A-04, and valid and rejected capability combinations address A-07. Figure 15 classifies this evidence.

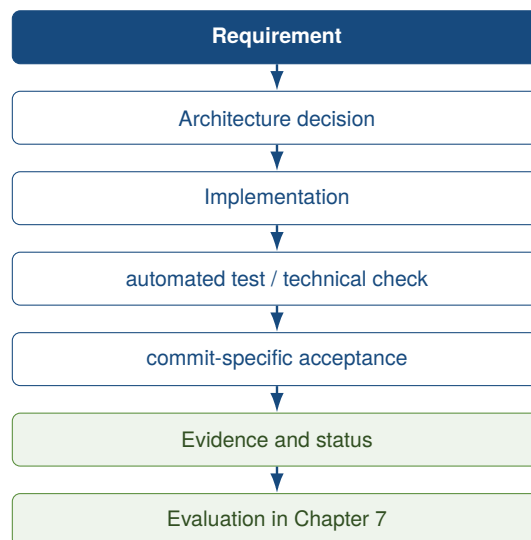


Figure 15: Evidence chain for technical verification

Source: Author's own illustration.

6.2 Continuous Integration

Core CI, Profile Matrix, PostgreSQL Integration, and Desktop CI separately check the baseline, five profiles, PostgreSQL combinations, and native packages. Release Validation runs only manually or for version tags. All workflows have read permissions and use `tools/control.py` (kleiveist, 2026c).

The runs dated 7 August and queried on 9 August 2026 correspond to HEAD `a4487ac` and are not entirely green. Core CI succeeded for the backend, schema, frontend, web build, Compose, and both images; tooling, profiles, and configuration failed. In the profile matrix, both web profiles passed, while the three desktop profiles failed during the project test. PostgreSQL integration succeeded for

the master and web-cloud, but not for the two desktop combinations. Desktop CI packaged on Linux and macOS; Windows failed during frontend installation (GitHub, 2026a, 2026c, 2026d, 2026e).

Locally, on 9 August 2026, all 194 tooling tests as well as the schema, API, and SQLAlchemy tests passed on the same HEAD under Python 3.13.14. These local results neither supersede the failed external jobs nor provide evidence for the paths that were not executed because Node.js, Rust, and Docker were unavailable. Release Validation was not run for this commit.

6.3 Acceptance and Technical Verification

The final acceptance report rates commit 1cdfb6e as *PASS WITH OPEN EXTERNAL ITEMS* and *READY FOR CASE STUDY*. This applies to the case study baseline, not to production readiness or external deployments. On this commit, 189 tooling tests passed (kleiveist, 2026j).

The acceptance process generated and installed all five profiles, executed profile-dependent diagnostics, tests, and web builds, and packaged the three desktop profiles as Linux DEBs. With Compose available, strict validation passed. The three backend profiles additionally passed with PostgreSQL, including the connection, migrations, tests, and build; the desktop variants were packaged again. As expected, the two incompatible PostgreSQL combinations were rejected before the target was generated (kleiveist, 2026j).

Figure 16 and Table 5 distinguish local acceptance from current CI. The more recent failures do not invalidate the commit-bound acceptance, but they prevent its application to a4487ac as fully green remote evidence.

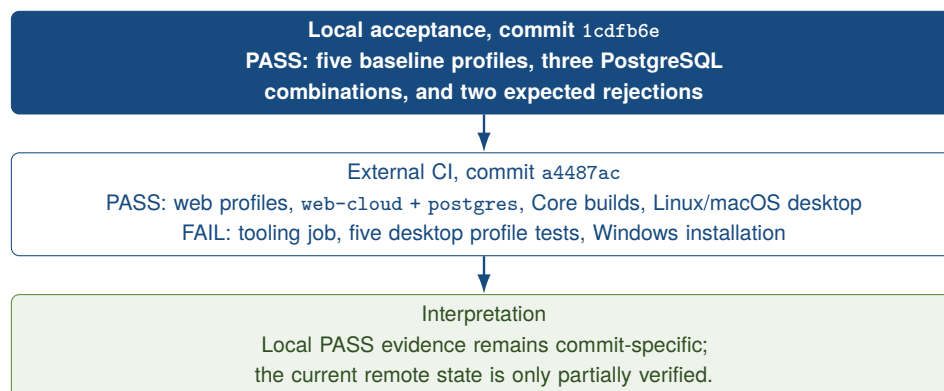


Figure 16: Profile verification by evidence state and commit

Source: Author's own illustration based on kleiveist (2026j) and the current GitHub Actions runs.

Table 5: Profile-specific technical verification

Profile / combination	Gen.	Install / Doctor	Tests	Web	Desktop	PG / migr.	CI on a4487ac
web-only	P	P	P	P	—	—	P
web-cloud	P	H	P	P	—	—	P
desktop-local	P	P	P	P	P (Linux DEB)	—	F: profile test
desktop-cloud	P	H	P	P	P (Linux DEB)	—	F: profile test
full-platform	P	H	P	P	P (Linux DEB)	—	F: profile test
web-cloud + postgres	P	P	P	P	—	P	P
desktop-cloud + postgres	P	P	P	P	P (Linux DEB)	P	F: profile test
full-platform + postgres	P	P	P	P	P (Linux DEB)	P	F: profile test

Legend: P = PASS; H = PASS WITH NOTE; F = executed check failed; — = not applicable. The six central result columns are taken from the local acceptance on 1cdfb6e; the final column reports the profile-specific GitHub Actions run on a4487ac. web-only + postgres and desktop-local + postgres were correctly rejected before target generation in the local acceptance (P). Source: kleiveist (2026j) and GitHub (2026d, 2026e).

6.4 Reproducibility

Two generations of desktop-cloud + postgres on 1cdfb6e were byte-for-byte identical for identical inputs. `init -dry-run` wrote nothing; nonempty targets and incompatible capabilities were rejected before writing. This evidence applies only to this combination and environment (kleiveist, 2026j; Lamb & Zacchiroli, 2022).

Frontend, Cargo, and three Python lockfiles pin dependencies; npm and Cargo use locked versions, and the container baseline additionally uses hashes. The hash-verified installation process was tested (kleiveist, 2026b, 2026c). The operating system, toolchain, registries, network, and platform SDKs nevertheless limit reproduction; native results remain platform-specific.

6.5 Security and Configuration Boundaries

Tooling tests provide evidence for configuration precedence and for validation of environments, ports, and CORS origins. They also check the profile-dependent `.env.example` and ensure that `DATABASE_URL` is present only for PostgreSQL. These tests passed locally, while the external tooling job failed on the same commit (GitHub, 2026a; kleiveist, 2026i).

Only `VITE_API_BASE_URL` may reach the frontend. Tests reject secrets designated as public, remove `DATABASE_URL` from the frontend and Tauri, and mask database passwords in diagnostics. No database variable is created without PostgreSQL. Additional checks cover health, readiness, error responses, database-independent liveness, the Tauri CSP, and capability metadata (kleiveist, 2026h, 2026j).

Thus, only the configuration, secret, and desktop boundaries of the baseline have been verified. Authentication, authorization, production credentials, TLS, and provider-specific hardening remain product-specific; a complete security assessment has not been demonstrated.

6.6 Outstanding External Checks

Table 6 distinguishes failed checks, missing evidence, and deliberate product boundaries. Failed CI jobs are real errors; a Compose stack that was not started, by contrast, is not verified.

The current Core job provides evidence for Compose validation and both master images, but not for either `docker compose up` or production deployment. Linux and macOS packages were built unsigned on a4487ac; the Windows package was not. Branch Protection could not be read because the required API permission was unavailable. Signing, notarization, updaters, production credentials, and authentication/RBAC require a product and external infrastructure (GitHub, 2026a, 2026c; kleiveist, 2026h).

Table 6: Outstanding and external verification items on a4487ac

Verification item	Status	Reason	Required evidence
Tooling and desktop-profile CI	FAIL	current jobs terminate in executed test steps with exit code 1	successful repetition on the corrected commit
Windows package	FAIL	frontend installation failed in the native Windows job	successful native build and artifact upload
Release Validation	not verified	no manual or tag run is available	successful non-publishing run on a fixed commit
Compose startup	not verified	only the model and master images were checked in CI	<code>docker compose up</code> , health/readiness evidence, and controlled teardown
Branch Protection	not verified	the public API requires authentication	authorized evidence of the required checks for <code>main</code>
Signing, notarization, updater	outside the scope	product-specific identity and protected credentials are intentionally absent	protected native release pipeline for each target platform
Production operation and auth/RBAC	outside the scope	cloud environment, business model, and security model are product-specific	product-specific acceptance including credentials and deployment

Source: Author's own compilation based on kleiveist (2026h, 2026j) and the current GitHub Actions runs.

7 Evaluation of the Technology Template

7.1 Productivity Effects

The evaluation compares the requirements, implemented mechanisms, and evidence. The criteria are reuse, standardization, variability, maintainability, testability, reproducibility, developer experience, extensibility, and platform coverage. Complexity, operational maturity, and documentability supplement this assessment. The project's internal acceptance report provides project-specific evidence, but no independent replication (kleiveist, 2026j). This distinction keeps observed mechanisms, executed verification activities, and inferred effects separate. Automation alone is not considered a measurable productivity gain.

Here, productivity encompasses the effort required to obtain a runnable initial project, recurring setup decisions, the development flow, and subsequent maintenance. A single activity metric does not represent it adequately (Forsgren et al., 2021). Conceptually, the effect can therefore be expressed as

$$\text{Productivity benefit} = \text{avoided project effort} - \text{template maintenance effort} - \text{additional complexity}$$

In the absence of time and effort measurements, it cannot be quantified. The relationship also shows that centralized maintenance and additional complexity can reduce the project effort avoided.

At project initialization, `control.py` combines profile selection, generation, system checks, and installation. The acceptance results provide evidence for these steps across all five profiles and for the rejection of incompatible PostgreSQL selections (kleiveist, 2026j). The mixed CI results for the later HEAD limit the transferability of this evidence. Less manual setup is therefore plausible, but a specific amount of time saved has not been demonstrated.

During development, profile-dependent entry points standardize startup, testing, and builds. The shared frontend, configuration contract, and clear responsibilities encourage reuse; specialized tools and platform prerequisites remain necessary. Central changes benefit future projects, but require shared maintenance of profiles, generator, tests, and documentation. Projects that have already been generated are not updated automatically. The benefit is thus distributed across project initialization, ongoing development, and subsequent baseline maintenance. It is particularly plausible when several technically related projects use the same foundations.

Quantitative evidence is lacking. A future controlled evaluation would need to compare similar projects with and without the template in terms of initialization and onboarding time, setup errors, failed builds, proportion of reuse, and maintenance time. It should also record the time to the first domain-specific feature and the number of manual setup decisions. Only such comparative data would permit a statement about the magnitude of the effect.

7.2 Strengths

The shared baseline combines centralized maintenance with clear boundaries between the frontend, backend, desktop, persistence, and tooling layers. The shared frontend addresses A-02 and A-05 without burdening every profile with all components. This separation supports independent tests and product-specific extensions without duplicating the shared presentation layer.

Five profiles represent recurring platform configurations. Dependencies prevent the use of PostgreSQL without a backend. The acceptance results for all base profiles and for the permitted and prohibited PostgreSQL combinations support this model for the limited current catalog, but not for an arbitrary number of future capabilities (kleiveist, 2026j).

`control.py` standardizes the entry point while keeping the specialized tools visible. The manifest, configuration contract, tests, and build paths increase traceability. The byte-identical generation of two `desktop-cloud + postgres` projects provides concrete evidence of this, albeit limited to one combination and environment (kleiveist, 2026j). Profile-dependent skipping of absent layers keeps the shared command interface aligned with the actual project scope.

Table 7 contrasts the strengths with the limits of their applicability.

7.3 Weaknesses and Limitations

Technical weaknesses, deliberate scope boundaries, and unverified areas must be distinguished. Only the first category directly denotes a disadvantage of the implementation. Although scope boundaries create a need for adaptation, they do not constitute an unmet requirement. A lack of verification

Table 7: Evidence-based comparison of strengths and limitations

Area	Strength and evidence	Limitation and evidence	Significance
Architecture	Separate frontend, backend, Tauri, and persistence boundaries; a shared frontend.	Shared schemas are not consumed automatically by the frontend and backend.	Reuse with clear responsibilities; contract changes remain manual.
Variability	Five declarative profiles; dependencies and three permitted PostgreSQL combinations have passed acceptance.	Only one selectable capability; the direct CLI does not strictly enforce the distinction between <code>optional</code> and <code>selectable</code> .	Controlled selection in the current catalog, with limited implications for extensions.
Tooling	Shared profile-dependent entry point for diagnostics, installation, startup, testing, and building.	Several specialist tools and native prerequisites remain necessary.	Fewer distinct interaction paths, but no technology-independent workflow.
Quality	Tests, the profile matrix, build paths, and the release gate are recorded in the acceptance report.	Current tooling, desktop-profile, and Windows jobs fail; Branch Protection has not been demonstrated.	A sound local verification basis, but no consistently successful remote evidence.
Reproducibility	Two identically configured projects were byte-for-byte identical in the documented experiment.	Evidence covers only one combination and one examination environment.	Specific evidence without claiming complete cross-platform reproducibility.
Operation and platforms	Web and image builds, PostgreSQL integration, and unsigned Linux/macOS packages are documented.	Compose startup, production cloud operation, a Windows package, signing, and notarization remain outstanding.	Template maturity must not be equated with production readiness.
Maintenance	Core, profiles, tooling, tests, and documentation reside in one baseline.	Changes may affect multiple profiles; product projects receive no automatic updates.	Central maintenance is consolidated, but requires broad regression testing and transfer processes.

Source: Author's own evaluation based on Chapters 3–6, the acceptance report, and the current CI runs (GitHub, 2026a, 2026c, 2026d, 2026e; kleiveist, 2026j).

does not yet permit either a positive or a negative technical judgment.

Consolidation in the master repository increases internal complexity: changes to the core, generator, or catalog require regression tests across several profiles. In addition, the frontend and backend do not use the shared JSON schemas at runtime; contract changes remain manual. Direct CLI selection distinguishes `optional` and `selectable` less clearly than the dialog does. Both discrepancies create risks of maintenance errors or incorrect operation. The examined HEAD also has conflicting evidence: the web profiles and the local tooling run passed, while the external tooling, desktop-profile, and Windows checks failed. This finding does not retroactively call the earlier, commit-specific acceptance into question, but it prevents a fully successful assessment of the more recent state.

The deliberate exclusions are a backend in `web-only`, a cloud API in `desktop-local`, and PostgreSQL without a backend. Domain logic, authentication, role models, mandatory persistence, cloud providers, and native domain-specific commands likewise remain product-specific. The resulting adaptation effort is not a deviation from the scope.

Compose startup, production deployment, Release Validation, and Branch Protection have not been conclusively verified. Unsigned Linux and macOS packages were built, while Windows packaging failed. Signing, notarization, publication, and production cloud operation remain product- or infrastructure-specific (GitHub, 2026a, 2026c; kleiveist, 2026j). The packaging results therefore demonstrate neither signed releases nor production cloud operation. The subject of the evaluation is the template baseline, not the production readiness of a product. The remaining risks therefore primarily concern operations, platforms, and release processes.

7.4 Maintenance Effort

The single-repository strategy consolidates the core, features, profiles, generator, tests, and documentation. This avoids parallel baseline changes and benefits future projects. Repositories that have already been generated must, however, incorporate fixes independently. The source of truth thus reduces the number of starting points that require maintenance, but not the effort needed to migrate existing products.

Central maintenance encompasses several languages, dependency managers, toolchains, platforms, and release cycles. This diversity follows from the web, backend, desktop, and deployment objectives, but increases the expertise required, the breadth of testing, and dependency on the environment. Native builds and external services are more maintenance-intensive than a purely web-based, technologically homogeneous starter.

Five profiles, six features, and one selectable capability currently limit the combination space; acceptance covers five base paths and three permitted PostgreSQL extensions (kleiveist, 2026j). Additional capabilities expand both the dependencies and the test matrix. A combinatorial explosion has not been demonstrated, but increasing maintenance effort is plausible. With such extensions, the catalog, resolver, configuration contract, test matrix, and documentation must remain mutually consistent.

Central maintenance is therefore more demanding than maintenance of a small, homogeneous starter, but can consolidate repeated maintenance of the baseline. Its economic viability depends on frequency of use, similarity between projects, and maintenance time; cost data are unavailable.

7.5 Comparison with Alternative Template Strategies

The architectural strategies are compared in terms of reuse, consistency, variability, entry complexity, maintenance, testability, and adoption of updates (Clements & Northrop, 2001; Krueger, 1992). The qualitative levels do not form an overall ranking; high entry complexity, for example, is a disadvantage.

Separate template repositories remain individually manageable, but can diverge. A *copy-and-paste starter* minimizes template logic, but provides no channel through which subsequent improvements can be propagated. GitHub templates accordingly create a separate, unrelated history (GitHub, 2026b). Separate repositories therefore give greater weight to organizational isolation than to the central consistency of shared components.

Framework-specific starters such as `create-vite` offer a simple entry point into a narrow area. The backend, desktop layer, persistence, and overall workflow must be added separately when needed (VoidZero Inc. and Vite Contributors, 2026). For a small, homogeneous frontend, this lower entry complexity may be more appropriate than the full-stack approach under examination.

A *monolithic universal template* simplifies selection, but burdens small projects with unused components. The examined *single repository with profiles and capabilities* combines centralized maintenance with selected paths. This increases the complexity of the master repository, resolver, and test matrix; updates do not reach generated projects automatically. It thus combines centralized variability before generation with the autonomy of a copied starter afterward.

The profile-based approach is particularly suitable for recurring, technically related projects. Small individual projects may benefit from framework starters, while organizationally separate product lines

Table 8: Qualitative comparison of template strategies

Strategy	ongoing reuse	Consistency	Variability	Entry complexity	Maintenance consolidation	Updates to existing projects
Separate template repositories	medium	medium	medium	low	low	manual for each template or project
Copy-and-paste starter	low after copying	low	high	low	low	no automatic adoption
Framework-specific starter	high within a narrow scope	high within a narrow scope	low to medium	low	high within the starter	usually manual after generation
Monolithic universal template	medium	high	low	high	high	depends on the selected derivation method
Single repository with profiles	high within the defined scope	high	high, controlled	medium	high	centralized in the master, manual in derived projects

Source: Author's own qualitative comparison based on Krueger (1992), Clements and Northrop (2001), GitHub (2026b), and VoidZero Inc. and Vite Contributors (2026).

may benefit from separate repositories. There is no universal winner, but rather a set of different trade-offs.

7.6 Overall Evaluation

Table 9 distinguishes criteria that are largely or partially addressed from those that have not been conclusively demonstrated. These levels are qualitative and measure neither quality nor productivity numerically.

Table 9: Consolidated evaluation of the technology template

Criterion	Evidence base	Assessment	Key limitation
Reuse and standardization	Shared Core, five profiles, generator, and uniform command interface	largely addressed	Applies to the defined stack; derived projects are not updated automatically.
Variability and extensibility	Profile presets, capability dependencies, and verified PostgreSQL combinations	largely addressed	Small current catalog; extensions increase the maintenance and testing scope.
Testability and reproducibility	Documented profile, integration, and build acceptance, plus a byte-for-byte identical generation comparison	partially addressed	Some current remote jobs failed; reproducibility experiment covers only one combination.
Developer experience and documentability	<code>control.py</code> , diagnostic paths, versioned manifests, and structured documentation	partially addressed	Onboarding and interaction effort were not measured empirically; multiple toolchains remain visible.
Maintainability	One central baseline with shared tests and documentation rules	partially addressed	Technology diversity, variant regression, and manual interface synchronization.
Platform coverage and operational maturity	Evidence covering web, backend, PostgreSQL, container images, Linux, and macOS; provider-neutral boundary	not conclusively demonstrated	Windows package, Compose startup, signing, notarization, and production deployment have not been evidenced.

Source: Author's own evaluation based on the preceding chapters, kleiveist (2026j), and the CI runs evaluated in Section 6.2.

The baseline is technically viable for the defined profiles. The core, presets, dependencies, and tooling support reuse, standardization, and flexibility. The acceptance results support generation, installation, tests, builds, PostgreSQL combinations, and Linux packaging (kleiveist, 2026j). More recent external runs add evidence for web, container, Linux, and macOS paths, but include failures in tooling, desktop-profile, and Windows jobs. The evidence is therefore robust, but varies in scope by commit and platform.

Limitations include the complexity of central maintenance, manually synchronized contracts, and the absence of updates for generated projects. Windows packaging, signed releases, notarization, and production cloud operation have not been demonstrated. Deliberately omitted provider and domain logic, by contrast, is not a defect. Table 9 makes the distinction between an addressed requirement and the outstanding scope of verification explicit.

Automation and shared workflows provide plausible, but unquantified, potential for productivity gains. The current state is suitable as a standardized foundation for the intended project families, but not as a finished production application or a universal template. Whether the central investment is economically worthwhile remains unresolved in the absence of usage and cost data.

8 Application Scenarios and Transfer to Customer Projects

8.1 Suitable Project Types

Suitability depends on access channels, the backend and persistence, deployment, native integration, and infrastructure, security, and operational needs. The evidence covers combinations that can be generated and verified profile paths. By contrast, the levels *well suited*, *conditionally suited*, and *not primarily intended* qualitatively assess the distance from the baseline, not the probability that a project will succeed (kleiveist, 2026g, 2026j). The more central requirements lie outside the available components and evidence, the greater the adaptation effort and remaining risk.

`web-only` is suitable for browser-based frontends without a template backend. `web-cloud` adds a central API and provider-neutral deployment structure, for example for internal business and data applications. PostgreSQL remains optional; authentication, authorization, domain models, and production infrastructure are product-specific. The profile therefore covers the initial technical structure, not the particular business application.

`desktop-local` is appropriate for local tools without a central backend, provided that native functions can be added within a manageable scope; a local database is absent. `desktop-cloud` adds central services, and `full-platform` also adds the browser channel. The primary application domain therefore comprises web-, cloud-, and desktop-oriented business and full-stack applications. For all desktop variants, more extensive native functions must be added and verified specifically.

Category	Project types	Decisive criterion
WELL SUITED	Browser frontend; web + backend/cloud; local desktop tool; desktop + cloud; web + desktop + cloud	Access channels and runtime layers match a verified profile; domain-specific and operational functions still need to be added.
CONDITIONALLY SUITED	high proportion of native code; highly customized infrastructure; stringent compliance or security requirements	A profile covers only part of the requirements; substantial extensions or additional evidence are required.
NOT PRIMARILY INTENDED	hard real-time systems; game-engine-based games; native mobile apps; core firmware and embedded systems	The underlying runtime or platform architecture falls outside the Vite/FastAPI/Tauri baseline.

Figure 17: Qualitative suitability overview by architectural fit and adaptation effort

Source: Author's own transfer assessment based on the profile definitions and technical acceptance (kleiveist, 2026g, 2026j).

Figure 17 and Table 10 summarize the classification. *Suitable* means only that the baseline covers essential platform components. Domain logic, user interface, integrations, product security, and operations remain open, as does the economic assessment.

Table 10: Mapping of technical project types to the template baseline

Project type	Suitability	Profile	Required additions and limitations
Browser frontend	well suited	web-only	Domain logic and UI; external APIs are possible, but the baseline provides no FastAPI backend.
Web application with API/cloud	well suited	web-cloud	Authentication, domain models, and production operation; PostgreSQL only if relational persistence is required.
Local desktop tool	well suited	desktop-local	Native commands, packaging, and signing are product-specific; no local database is included.
Desktop application with central backend	well suited	desktop-cloud	Production API, security, and signing; PostgreSQL remains optional.
Browser + desktop + cloud	well suited	full-platform	Channel-specific UX and tests; persistence only with a separate capability.
Application with a high proportion of native code	conditionally suited	desktop profile or another foundation	Substantial effort for platform APIs, permissions, system-level services, or a native UI.
Highly specialized infrastructure	conditionally suited	depends on the partial architecture	Suitable only if the client or an individual service can be meaningfully isolated; infrastructure is not prescribed.
Hard real-time systems, game-engine-based games, native mobile applications, or core embedded systems	not primarily intended	no profile	The underlying runtime and platform requirements fall outside the baseline.

Source: Author's own qualitative mapping based on kleiveist (2026g) and kleiveist (2026j).

8.2 Unsuitable and Conditionally Suitable Project Types

Projects requiring extensive operating-system integration, a platform-specific UI, system-level services, or drivers are only conditionally suited. The Tauri shell does not provide sufficient evidence

of an adequate native foundation for such projects. Highly distributed infrastructure and specialized event-streaming or HPC requirements may likewise require a different architecture; at most, the baseline remains usable for a clearly delimited part.

Hard real-time systems, graphically intensive games, firmware, embedded systems, and native mobile applications do not belong to the primary target domain. Ordinary live updates do not turn a business application into a real-time system. Individual mobile or hardware-oriented functions are not excluded, but the template does not provide their technical foundation. The decisive question is therefore not whether a single characteristic is present, but whether the baseline supports the technical core of the product.

High security or compliance requirements do not preclude use of the baseline, but require additional evidence concerning protection requirements, audits, certification, and legal compliance. Compose validation and master images are evidenced, but Compose startup and production deployment are not (GitHub, 2026a). Linux and macOS packages were built without signing; the Windows packaging path failed (GitHub, 2026c). Signing and notarization remain product-specific (kleiveist, 2026h). The required TypeScript, Python, Rust, database, and deployment expertise must also be assessed. If this evidence or expertise is unavailable, the project is at most conditionally suited.

8.3 Analysis of Customer Requirements

The analysis begins with the functional objective, user groups, workflows, and deployment environment. Only then are the requirements classified in technical terms. Shared data processing, for example, may require a backend and persistence, but does not yet determine the use of PostgreSQL.

Browser and desktop access, server functions, persistence, offline needs, native functions, external APIs, operations, deployment, security, and compliance are recorded separately. Despite lacking a cloud backend, `desktop-local` includes neither a local data model nor synchronization. Table 11 consolidates the guiding questions.

Table 11: Compact checklist for technology-neutral requirements analysis

Area	Guiding question	Influence on selection
Domain functionality and use	Which workflows, user groups, and locations of use must be supported?	Determines the product scope; does not yet determine a profile.
Access channels	Will the product be used in a browser, as a desktop application, or through both channels?	Distinguishes web, desktop, and full-platform paths.
Central services	Must functions or shared data be available on the server side?	Requires a backend-capable cloud profile.
Persistence and offline operation	Is centralized relational data required; must the client remain operational without a network connection?	PostgreSQL only when a backend is required; local storage and synchronization are separate additions.
Native integration	Which operating-system APIs, devices, or platform-specific interfaces are required?	Determines the need for Tauri and the distance from the native baseline.
Integration and operation	Which external systems, target environments, and scaling and operational requirements exist?	Produces product-specific adapters and infrastructure decisions.
Security and compliance	Which protection, evidence, audit, or regulatory requirements apply?	Requires additional measures and evidence independently of the profile.

Source: Author's own compilation.

The result comprises platform components and gaps, for example in authentication, data models,

integrations, observability, backups, infrastructure, or signing. Only then can a decision be made between a suitable profile, a limited extension, and a solution outside the baseline. The method therefore deliberately follows the sequence of requirements, technical classification, and only then profile or technology selection.

8.4 Selecting the Project Profile

The access channels determine the profile: `web-only` denotes a browser without a backend, `web-cloud` a browser with a backend, `desktop-local` a desktop application without a cloud backend, `desktop-cloud` a desktop application with a backend, and `full-platform` the browser and desktop channels together. There is no profile for a backend without provider-neutral deployment files.

The smallest suitable profile is selected. PostgreSQL is then assessed as a separate dimension and added to a backend profile only when needed. This is not permitted for `web-only` and `desktop-local` (kleiveist, 2026g).

The documentation records whether the profile is a complete match, requires specified extensions, or fails to meet central requirements. In the last case, a different architecture or clearly delimited partial use should be examined instead of selecting the largest profile. Team expertise and economic constraints supplement this technical decision, but do not change the meaning of the profiles.

8.5 Generating a Customer Project

`init` creates a separate project from the profile, capability, and project identity. Its active manifest is named `project-profile.toml`. Generation is the handover point to product development, not the completion of the product (kleiveist, 2026g, 2026l). For desktop projects, these inputs include a valid application ID.

In the target directory, `doctor`, `install`, `run`, and `test --suite all` check the baseline. Profile-dependent Tauri, database, or container checks supplement the transfer shown in Figure 18.

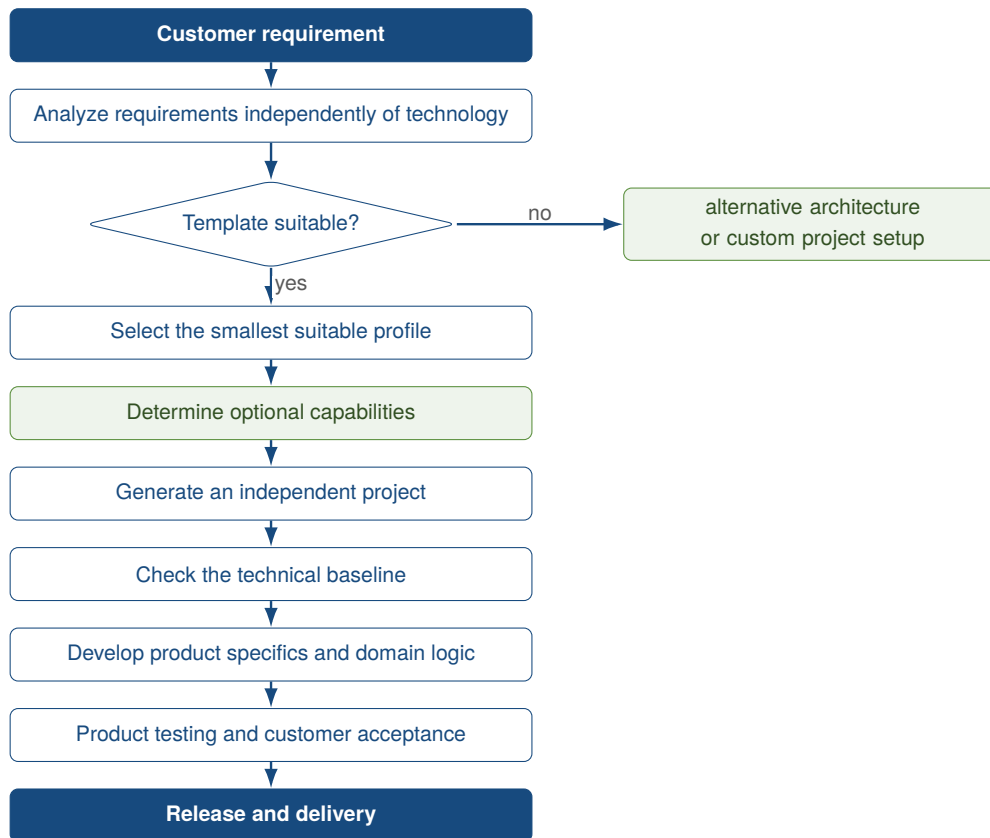


Figure 18: Vertical transfer process from customer requirement to product project

Source: Author's own illustration.

8.6 Transition to Product Development

After generation, the independent product project assumes responsibility for domain models, business rules, the user interface, integrations, data, and operations. Authentication, authorization, scaling, monitoring, backups, data protection, and signing also remain product tasks. The baseline structures this work, but does not solve these product-specific tasks.

The master repository and the product have separate lifecycles. There is no permanent update or synchronization mechanism. Changes to the master repository must be transferred to products deliberately; product changes should flow back into the baseline only after a generalization and verification review.

Template acceptance and product acceptance remain separate. The commit-specific acceptance provides evidence for generation, installation, tests, and selected baseline builds, but not for deployment, signing, or functional and security requirements (kleiveist, 2026j). The product requires its own tests and evidence for these areas. They must cover the specific customer requirements, integrations, and operating conditions.

9 Conclusion

9.1 Summary of Results

The master template addresses repeated setup effort and diverging project bases through a shared core and declarative variability. It consolidates generation, configuration, boundaries of responsibility, workflows, and maintenance.

Five profiles combine the shared Vite frontend with a profile-dependent FastAPI backend and an optional Tauri layer; PostgreSQL can be selected only with a backend. `control.py` standardizes the development path. Generated projects remain independent and receive no automatic updates from the master repository.

The local acceptance of an earlier commit supports the profile, PostgreSQL, test, web build, and Linux packaging paths. The current external CI is only partially successful; further evidence is either unavailable or product-specific. The evidence therefore demonstrates a baseline with controlled variability, not production readiness or empirical suitability for customer projects.

9.2 Answers to the Research Questions

Research Question 1: Requirements Addressed. The template addresses the functional and technical, quality, and operational requirements through a baseline that can be generated, clearly defined components, profiles, shared workflows, centralized maintenance, and versioned configuration. Separate runtime and verification paths support this; domain logic, authentication, authorization, and production operation remain product-specific.

Research Question 2: Variants for Web, Cloud, and Desktop. Five presets combine the shared frontend with backend, cloud, and Tauri components; PostgreSQL is added only when backend capability is present. Generated projects predominantly contain the paths they require. `desktop-cloud` and `full-platform` are scaffolded identically at the technical level and differentiated semantically.

Research Question 3: Reuse and Standardization. Centralized maintenance of implementations, profiles, the generator, configuration, tests, and documentation promotes reuse and consistent initial states across projects. The benefit applies to the baseline and future generations; existing product projects receive no automatic updates.

Research Question 4: Reduction of Recurring Effort. Automated or standardized profile selection, generation, diagnostics, installation, startup, testing, and builds provide plausible potential for reducing recurring effort. Time, effort, and comparative data are lacking, however; a productivity improvement has not been demonstrated when central maintenance and multiple toolchains are taken into account.

Research Question 5: Suitability for Project Types and Customer Requirements. The template is suitable as a foundation for related web, desktop, and combined business applications whose

platform, cloud, and persistence needs can be represented. Highly native applications, games, and specialized real-time systems lie outside the primary target domain. The qualitative classification is not a universal assurance of suitability.

Research Question 6: Limitations, Maintenance Effort, Risks, and External Dependencies.

Limitations include the growing breadth of regression testing, manual contract synchronization, insufficiently strict separation between optional and selectable capabilities, the absence of product updates, and the effort of maintaining multiple toolchains. The tooling, desktop-profile, and Windows CI paths are currently failing; Release Validation, Compose startup, and Branch Protection are still missing. Signing, notarization, the updater, production deployment, and the security model remain product tasks.

9.3 Critical Reflection

The project-specific case study has limited generalizability. The documentation, acceptance, and local verification originate from the project context; the developer perspective may influence the selection of evidence. Literature and official documentation are no substitute for independent replication. Commit-specific findings remain valid, but their transfer to other repository states or projects is limited.

The evidence is heterogeneous: an earlier acceptance performed predominantly on Linux contrasts with mixed current CI results. Without test-coverage measurement, Compose startup, and successful platform runs, there is no comprehensive evidence of quality or production readiness; unsigned packages are no substitute for product releases.

Three principles in particular are transferable: a centralized baseline, declarative variability, and shared tooling entry points. Productivity and suitability were assessed only qualitatively; time measurements, comparison groups, customer cases, and economic data are lacking. These gaps particularly limit statements about the actual effects in everyday project work.

9.4 Outlook

In the short term, the failing tooling, desktop-profile, and Windows CI paths should be corrected on a fixed commit. Compose startup with health and readiness checks, Release Validation, and Branch Protection should supplement the technical evidence without claiming production readiness.

In the medium term, specific products require signing, notarization, and an updater. Credentials, deployment, authentication, and RBAC also remain product-specific. Three measures are appropriate for the baseline: automate the use of shared contracts, make capability selection stricter, and synchronize generated projects in a controlled manner.

In the long term, a comparative study across projects should measure onboarding, time to the first build, setup errors, and maintenance effort. Real-world customer projects could test the profile and suitability criteria.

9.5 Conclusion

The template is a centrally maintained, profile-based baseline for the defined web, cloud, and desktop variants. The shared core, capabilities, and tooling standardize recurring project workflows.

Controlled variability provides plausible potential for productivity gains. Without empirical data, its magnitude and suitability for customer projects can be substantiated analytically only for the delimited target domain.

The evidence extends only to the examined technical baseline. The local acceptance, current CI failures, outstanding external verification activities, and product-specific release and security tasks do not permit it to be equated with a finished, cross-platform-verified product.

References

- Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Retrieved August 8, 2026, from <https://alembic.sqlalchemy.org/en/latest/tutorial.html>
- Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Retrieved August 8, 2026, from <https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/>
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The space of developer productivity: There's more to it than you think. *Queue*, 19(1), 20–48. <https://doi.org/10.1145/3454122.3454124>
- GitHub. (2026a, August 7). *Core ci, run 31168215013* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215013>
- GitHub. (2026b). *Creating a repository from a template* [GitHub Documentation]. Retrieved August 9, 2026, from <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template>
- GitHub. (2026c, August 7). *Desktop ci, run 31168214763* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/actions/runs/31168214763>
- GitHub. (2026d, August 7). *Postgresql integration, run 31168216208* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/actions/runs/31168216208>
- GitHub. (2026e, August 7). *Profile matrix, run 31168215737* [GitHub Actions run for commit a4487ac in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215737>
- ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard ISO/IEC 25010:2023). International Organization for Standardization. Retrieved August 8, 2026, from <https://www.iso.org/standard/78176.html>
- Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Retrieved August 8, 2026, from <https://doc.rust-lang.org/book/>
- kleiveist. (2026a, August 7). *Central project control cli* [CLI implementation in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/tools/control.py>
- kleiveist. (2026b, August 7). *Container builds and local production simulation* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/container-builds.md>
- kleiveist. (2026c, August 7). *Continuous integration* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/ci.md>
- kleiveist. (2026d, August 6). *Database feature* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/def/database-feature.md>

- kleiveist. (2026e, August 6). *Deployment architecture* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/deployment-architecture.md>
- kleiveist. (2026f, August 6). *Framework architecture* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026g, August 6). *Project profiles* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md>
- kleiveist. (2026h, August 7). *Release and desktop packaging model* [Project documentation in the Template-Projekte repository]. Retrieved August 9, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/release-model.md>
- kleiveist. (2026i, August 6). *Runtime configuration* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/configuration.md>
- kleiveist. (2026j, August 7). *Template final acceptance* [Technical acceptance report in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026k, August 7). *Template-projekte: Full-stack project template* [GitHub repository, examined commit a4487ac]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026l, August 6). *Tooling guide* [Project documentation in the Template-Projekte repository]. Retrieved August 8, 2026, from <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md>
- Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, 24(2), 131–183. <https://doi.org/10.1145/130844.130856>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073045>
- Microsoft. (2026, July 27). *The typescript handbook* [TypeScript Documentation]. Retrieved August 8, 2026, from <https://www.typescriptlang.org/docs/handbook/intro.html>
- PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Retrieved August 8, 2026, from <https://www.postgresql.org/docs/current/index.html>
- Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Retrieved August 8, 2026, from <https://fastapi.tiangolo.com/features/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- SQLAlchemy Authors and Contributors. (2026, June 15). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Retrieved August 8, 2026, from <https://docs.sqlalchemy.org/en/20/core/engines.html>
- Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Retrieved August 8, 2026, from <https://v2.tauri.app/security/capabilities/>

Tauri Contributors. (2026b, July 22). *What is tauri?* [Tauri 2 Documentation]. Retrieved August 8, 2026, from <https://v2.tauri.app/start/>

VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Retrieved August 8, 2026, from <https://vite.dev/guide/>